

DigiMenu-4B

Coding Standard Document

1. Pendahuluan

Dokumen ini menetapkan standar penulisan kode (Coding Standard) untuk proyek DigiMenu-4B, sebuah aplikasi web manajemen menu restoran digital berbasis Laravel. Standar ini wajib diikuti oleh seluruh anggota tim pengembang agar kode yang dihasilkan konsisten, mudah dibaca, mudah dipelihara, dan bebas dari celah keamanan yang umum.

1.1 Tujuan

- Menjaga konsistensi gaya kode di seluruh codebase.
- Mempermudah proses code review dan onboarding anggota baru.
- Mencegah bug umum dan celah keamanan sejak tahap penulisan kode.
- Memastikan maintainability jangka panjang proyek.

1.2 Ruang Lingkup

Standar ini mencakup semua kode yang ditulis dalam proyek DigiMenu-4B, meliputi:

- PHP (Controller, Model, Middleware)
- Blade Templates (Views)
- JavaScript / Vite
- CSS / Tailwind
- Database Migrations & Seeders
- Konfigurasi dan routing Laravel

1.3 Stack Teknologi

Komponen	Teknologi & Versi
Backend Framework	Laravel 12.x
Bahasa Pemrograman	PHP ^8.2
Template Engine	Blade (Laravel built-in)
Frontend Styling	Tailwind CSS
Build Tool	Vite
Database	MySQL / SQLite
ORM	Eloquent ORM
Authentication	Laravel Breeze / Auth scaffold
Testing	PHPUnit 11.x
Code Style Tool	Laravel Pint (PSR-12)

2. Konvensi Penamaan (Naming Conventions)

Konsistensi penamaan adalah kunci keterbacaan kode. Proyek ini mengikuti konvensi standar Laravel/PHP.

2.1 Kelas (Classes)

Gunakan `PascalCase` untuk semua nama kelas, termasuk Controller, Model, Middleware, dan Service.

Type	Contoh Benar
Controller	MenuController, MejaController, UsersController
Model	Menu, KategoriMenu, DetailOrder, OpsiMenu
Middleware	AdminMiddleware, KasirMiddleware
Migration class	CreateKategoriMenuTable
Seeder	MenuSeeder, UserSeeder

2.2 Metode dan Fungsi

Gunakan `camelCase` untuk semua nama metode.

// ☒ BENAR
public function index() { ... }
public function store(Request \$request) { ... }
public function updateStatus(Request \$request, \$id_menu) { ... }
public function showChangePassword(Request \$request) { ... }
// ☐ SALAH
public function Update_Status() { ... }
public function show_change_password() { ... }

2.3 Variabel

Gunakan `camelCase` untuk variabel PHP. Untuk variabel yang merepresentasikan koleksi, gunakan bentuk jamak (plural).

// ☒ BENAR - variabel tunggal
\$menu = Menu::findOrFail(\$id_menu);
\$meja = Meja::findOrFail(\$id_meja);

```
// ☒ BENAR - koleksi
$menus = Menu::all();
$users = User::all();
$orders = Order::where('status', 'pending')->get();

// ☐ SALAH
$Menu = Menu::all();
$get_meja = Meja::all();
```

2.4 Database: Tabel dan Kolom

Gunakan `snake_case` untuk nama tabel dan kolom, sesuai konvensi yang sudah diterapkan di project ini.

Elemen	Konvensi	Contoh dari DigiMenu-4B
Nama tabel	snake_case, plural atau sesuai entitas	menu, kategori_menu, detail_order
Primary key	id_{nama_tabel}	id_menu, id_kategori, id_meja
Foreign key	id_{tabel_referensi}	id_kategori, id_menu, id_kasir
Kolom biasa	snake_case, deskriptif	nama_item, no_meja, nama_kategori
Kolom status	snake_case	status, status_meja

2.5 Route Names

Gunakan format `resource.action` untuk penamaan route, sesuai yang sudah diimplementasi.

```
// ☒ Pola yang digunakan di proyek
Route::get('/', [MenuController::class, 'index'])->name('menu.index');
Route::post('/', [MenuController::class, 'store'])->name('menu.store');
Route::put('/update/{id_menu}', [MenuController::class, 'update'])->name('menu.update');
Route::delete('/{id_menu}', [MenuController::class, 'destroy'])->name('menu.destroy');
```

3. Standar Penulisan PHP

Proyek ini mengikuti PSR-12 Extended Coding Style, yang merupakan standar resmi komunitas PHP dan didukung oleh Laravel Pint.

3.1 Indentasi dan Spasi

- Gunakan 4 spasi untuk satu level indentasi (bukan tab).
- Baris kosong di antara metode dalam sebuah kelas.
- Tidak ada trailing whitespace di akhir baris.
- File PHP harus diakhiri dengan satu baris kosong.

```
class MenuController extends Controller
{
    public function index()
    {
        $menu = Menu::all();
        return view('menu.index', compact('menu'));
    }

    public function store(Request $request)
    {
        // ...
    }
}
```

3.2 Namespace dan Use Statements

- Namespace harus ada di baris pertama setelah tag pembuka PHP.
- Urutkan use statement secara alfabetis.
- Pisahkan use Laravel core, package eksternal, dan kelas aplikasi dengan baris kosong.

```
<?php

namespace App\Http\Controllers;

use App\Models\Menu;
use App\Models\KategoriMenu;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
```

3.3 Controller

Controller harus mengikuti prinsip Single Responsibility. Logika bisnis yang kompleks harus dipindahkan ke Service class.

Aturan Controller
Satu Controller bertanggung jawab atas satu resource (contoh: MenuController → resource 'menu').
Method CRUD standar: index, create, store, show, edit, update, destroy.
Selalu gunakan <code>findOrFail()</code> bukan <code>find()</code> untuk mengambil single record.
Kembalikan <code>redirect()->back()->with('success_message', '...')</code> setelah operasi tulis.
Jangan letakkan logika query kompleks langsung di Controller; pindahkan ke Model scope atau Service.
Jangan return data JSON dari Controller kecuali untuk endpoint API terpisah.

3.4 Model (Eloquent)

- Selalu definisikan `$table`, `$primaryKey`, dan `$fillable`.
- Jangan letakkan primary key di dalam `$fillable` kecuali ada alasan khusus.
- Definisikan semua relasi Eloquent (`belongsTo`, `hasMany`, dll.) di dalam Model.
- Hindari import yang tidak digunakan, seperti `use function Laravel\Prompts\table;` yang ditemukan di Model Menu.

```
// Contoh Model yang Baik (mengacu pada Menu.php)
class Menu extends Model
{
    protected $table      = 'menu';
    protected $primaryKey = 'id_menu';

    protected $fillable = [
        'id_kategori', 'nama_item', 'harga',
        'deskripsi', 'gambar', 'status',
    ];

    public function kategori(): BelongsTo
    {
        return $this->belongsTo(KategoriMenu::class, 'id_kategori',
            'id_kategori');
    }
}
```

3.5 Validasi Request

Semua input dari pengguna WAJIB divalidasi sebelum diproses. Gunakan metode `validate()` bawaan Laravel.

Aturan Validasi
Selalu validasi setiap field yang diterima dari request.
Gunakan aturan validasi yang spesifik (integer, min:0, in:kosong,terisi, dll.).
Untuk upload gambar, gunakan aturan 'image' (bukan 'img' – ini typo yang perlu diperbaiki).
Gunakan Form Request class untuk validasi kompleks agar Controller tetap ramping.
Jangan pernah langsung menyimpan <code>\$request->all()</code> tanpa validasi.

Catatan Bug: Di `MenuController@store`, aturan validasi `'gambar' => 'required|img'` menggunakan `img` yang tidak valid. Harus diganti dengan `'required|image|mimes:jpg,jpeg,png,webp|max:2048'`.

4. Standar Routing

4.1 Pengelompokan Route

Semua route yang memerlukan autentikasi harus dimasukkan ke dalam grup middleware `auth`, seperti yang sudah diterapkan di `web.php`.

```
// Pola yang benar (web.php)
Route::group(['middleware' => ['auth']], function () {
    Route::prefix('menu')->name('menu.')->group(function () {
        Route::get('/', [MenuController::class, 'index'])->name('index');
        Route::post('/', [MenuController::class, 'store'])->name('store');
        Route::put('/update/{id_menu}', [MenuController::class, 'update'])->name('update');
        Route::patch('/updateStatus/{id}', [MenuController::class, 'updateStatus'])->name('updateStatus');
        Route::delete('/{id_menu}', [MenuController::class, 'destroy'])->name('destroy');
    });
});
```

4.2 Aturan HTTP Method

Operasi	HTTP Method	Contoh Route
Menampilkan daftar	GET	GET /menu
Menambah data baru	POST	POST /menu
Memperbarui semua field	PUT	PUT /menu/update/{id}
Memperbarui sebagian field	PATCH	PATCH /menu/updateStatus/{id}
Menghapus data	DELETE	DELETE /menu/{id}

4.3 Middleware pada Route

- Gunakan middleware `auth` untuk semua route yang memerlukan login.
- Gunakan middleware `kasir` hanya untuk aksi yang eksklusif kasir (misal: `updateStatus`).
- Gunakan middleware `admin` untuk aksi manajemen user.
- Jangan biarkan route sensitif tanpa middleware proteksi.

```
// Contoh penerapan middleware spesifik
Route::patch('/updateStatus/{id_menu}', [MenuController::class, 'updateStatus'])
    ->name('updateStatus')
    ->middleware('kasir');
```

4.4 Bug yang Ditemukan di Routing

Ditemukan typo pada route update menu: `'/update/{id_men\n u}'` — ada line break di dalam string URI. Ini harus diperbaiki:

```
// SALAH (bug aktif di web.php)
Route::put('/update/{id_men
    u}', [MenuController::class, 'update'])->name('update');
```

```
// ☒ BENAR
Route::put('/update/{id_menu}', [MenuController::class, 'update'])->name('update');
```


5. Standar Database & Migration

5.1 Migration

- Setiap tabel harus dibuat melalui migration, bukan langsung di database.
- Selalu sertakan method `up()` dan `down()` di setiap migration.
- Gunakan tipe kolom yang tepat dan deskriptif.
- Definisikan foreign key constraint secara eksplisit menggunakan `$table->foreign()`

Type Data	Penggunaan	Contoh di Proyek
<code>id('nama_pk')</code> <code>string()</code> <code>text()</code> <code>decimal(8,2)</code>	Primary key custom Teks pendek (VARCHAR) Teks panjang Harga/uang	<code>id('id_menu')</code> , <code>id('id_kategori')</code> <code>nama_item</code> , <code>nama_kategori</code> , <code>email</code> deskripsi harga (gunakan decimal bukan integer)
<code>integer()</code>	Angka bulat	<code>no_meja</code> , <code>kapasitas</code>
<code>enum()</code>	Nilai terbatas	<code>role</code> ['admin','kasir'], <code>status</code> ['kosong','terisi']
<code>boolean()</code> <code>timestamps()</code>	True/False <code>created_at</code> & <code>updated_at</code>	<code>status_menu</code> (aktif/tidak) Semua tabel utama

5.2 Eloquent Relationships

Selalu definisikan relasi di kedua sisi (Model induk dan anak) sesuai kebutuhan.

```
// Menu memiliki banyak OpsiMenu
public function opsiMenu(): HasMany
{
    return $this->hasMany(OpsiMenu::class, 'id_menu', 'id_menu');
}

// OpsiMenu milik Menu
public function menu(): BelongsTo
{
    return $this->belongsTo(Menu::class, 'id_menu', 'id_menu');
}
```

6. Standar Keamanan (Security)

6.1 Validasi & Sanitasi Input

- Semua input dari form HARUS divalidasi menggunakan Laravel Validator.
- Gunakan `$request->only([...])` bukan `$request->all()` saat menyimpan data.
- Blade templates secara otomatis meng-escape output dengan `{{ }}`, gunakan selalu kecuali ada keperluan spesifik.
- Jangan gunakan `{!! !!}` kecuali data sudah dipastikan aman dan di-sanitasi.

6.2 CSRF Protection

Semua form POST/PUT/PATCH/DELETE WAJIB menyertakan `@csrf` directive di Blade template.

```
<form method='POST' action='{{ route("menu.store") }}'>
    @csrf
    <!-- form fields -->
</form>

{{-- Untuk method PUT/PATCH/DELETE, tambahkan juga: --}}
@method('PUT')
```

7. Standar Blade Templates (Views)

7.1 Struktur dan Organisasi View

- Organisasikan view dalam subfolder berdasarkan resource: `resources/views/menu/`, `resources/views/meja/`, dll.
- Gunakan Blade layout dengan `@extends` dan `@section` untuk menghindari duplikasi HTML.
- Komponen yang digunakan berulang harus dijadikan Blade partial di `resources/views/partials/`.

```
{{-- resources/views/menu/index.blade.php --}}
@extends('layouts.app')

@section('title', 'Daftar Menu')
```

```
@section('content')
    @include('partials.alert')
    {{-- konten halaman --}}
@endsection
```

7.2 Menampilkan Data

- Selalu gunakan `{{ $variable }}` untuk output (auto-escaped).
- Gunakan `{{ number_format($menu->harga, 0, ',', '.') }}` untuk memformat harga.
- Gunakan `@if` / `@foreach` / `@forelse` dengan konsisten dan tutup selalu dengan `@endif` / `@endforeach` / `@endforelse`.

```
@forelse ($menu as $item)
    <tr>
        <td>{{ $item->nama_item }}</td>
        <td>Rp {{ number_format($item->harga, 0, ',', '.') }}</td>
    </tr>
@empty
    <tr><td colspan='5'>Tidak ada data menu.</td></tr>
@endforelse
```

8. Standar Middleware

8.1 Implementasi Middleware

Proyek ini memiliki dua custom middleware: AdminMiddleware dan KasirMiddleware. Keduanya memeriksa role user setelah autentikasi.

Standar Penulisan Middleware

Cek `Auth::check()` sebelum mengakses `Auth::user()` untuk mencegah error.

Kembalikan `redirect()->back()` dengan pesan error yang informatif saat akses ditolak.

Daftarkan semua custom middleware di `bootstrap/app.php` (Laravel 12) atau `Kernel.php`.

Beri nama alias middleware yang jelas: 'admin', 'kasir'.

Jangan menaruh logika bisnis di dalam Middleware – tugasnya hanya filter request.

8.2 Pendaftaran Middleware

Di Laravel 12, middleware didaftarkan melalui `bootstrap/app.php`. Pastikan alias konsisten dengan yang digunakan di routes.

```
// bootstrap/app.php
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'admin' => \App\Http\Middleware\AdminMiddleware::class,
        'kasir'  => \App\Http\Middleware\KasirMiddleware::class,
    ]);
});
```

9. Komentar dan Dokumentasi Kode

9.1 Kapan Menggunakan Komentar

- Komentar menjelaskan MENGAPA, bukan APA. Kode yang baik sudah menjelaskan dirinya sendiri.
- Tambahkan PHPDoc block pada semua method publik di Controller dan Model.
- Komentar satu baris untuk logika yang tidak intuitif.
- Komentar dalam Bahasa Indonesia diperbolehkan sesuai konteks proyek.

9.2 Format PHPDoc

```
/**
 * Memperbarui status menu (aktif/nonaktif).
 *
 * @param \Illuminate\Http\Request $request
 * @param int $id_menu
 * @return \Illuminate\Http\RedirectResponse
 */
public function updateStatus(Request $request, int $id_menu): RedirectResponse
{
    $request->validate(['status' => 'required|boolean']);
    $menu = Menu::findOrFail($id_menu);
    $menu->update($request->only('status'));
    return redirect()->back()->with('success_message', "Status menu diperbarui.");
}
```

```
}
```

10. Standar Testing

10.1 Struktur Test

Type Test	Lokasi	Cakupan
Unit Test	tests/Unit/	Model, helper functions, logika bisnis terisolasi
Feature Test	tests/Feature/	HTTP request, route, controller, middleware
Integration Test	tests/Feature/	Interaksi antar komponen termasuk DB

10.2 Aturan Testing

- Setiap Controller method harus memiliki minimal satu Feature test.
- Test harus berjalan di environment terpisah (database SQLite in-memory).
- Gunakan `RefreshDatabase` trait untuk memastikan kondisi database bersih tiap test.
- Beri nama test yang deskriptif: `test_user_can_create_menu`, `test_kasir_cannot_access_user_management`.

```
class MenuTest extends TestCase
{
    use RefreshDatabase;

    public function test_admin_can_create_menu(): void
    {
        $admin = User::factory()->create(['role' => 'admin']);
        $this->actingAs($admin)
            ->post(route('menu.store'), [...])
            ->assertRedirect();
        $this->assertDatabaseHas('menu', ['nama_item' => 'Nasi Goreng']);
    }
}
```